

DAPPER: Divide-and-Plan Pedagogical Reasoning for Educational Question Answering

Abstract. Large Language Models (LLMs) have shown remarkable capabilities in educational question answering, with strong performance in generating accurate and coherent final answers. However, their practical value in educational settings remains limited when students and instructors cannot inspect, interpret, or verify the reasoning behind such answers. This challenge is central to EXACT 2026, which evaluates systems on logic-based educational queries and physics reasoning problems with an emphasis on explainable outputs. We propose DAPPER, a divide-and-plan pedagogical reasoning approach for transparent educational question answering. For logic-based queries, DAPPER decomposes natural-language inputs into structured logical representations and derives answers through Z3-based entailment and refutation. For physics problems, it first generates an interpretable solution plan, then produces executable symbolic code that is safely executed and iteratively debugged. Across both tasks, LLMs are used for parsing, planning, and explanation, while solver-based inference and program execution provide the final reasoning basis. This design yields reasoning artifacts, including formalized premises, solver-derived reasoning traces, executable code, and plan-level explanations. In the official Round 2 evaluation, DAPPER ranked second overall, demonstrating that execution-grounded reasoning can improve transparency while maintaining competitive performance in educational question answering.

Keywords: Explainable question answering · Neuro-symbolic reasoning · Physics reasoning · Logic reasoning

1 Introduction

Large Language Models (LLMs) have achieved significant success in question-answering tasks, particularly within the field of education [1]. However, a plausible-sounding answer is not necessarily a correct or verifiable solution. When an LLM provides only the final answer without showing the solution process, it is difficult for teachers or users to understand the model’s reasoning, determine whether it correctly utilized the problem’s data, or identify at which step an error occurred if the answer is incorrect.

This issue is particularly evident in multi-step problems. For instance, in academic advising or the processing of academic regulations, the system must consider various conditions and rules before generating a result. In physics problems, the system needs to select the correct formula, handle units properly, and

perform calculations accurately. In both cases, the answer might appear plausible yet still contain errors; without insight into the solution process, it is very difficult to detect such mistakes by looking only at the final result.

EXACT 2026 addresses this setting by evaluating both answer quality and the clarity of the underlying reasoning. Organized in conjunction with IJCNN, the challenge includes policy-style questions that require relevant rule support and physics questions, such as electric-circuit problems, that require an understandable path to the final value.

Symbolic reasoning offers one way to make this process explicit, either as a standalone engine or as a component alongside an LLM. EXACT 2026 does not prescribe a particular technique; instead, it provides a setting in which symbolic solvers, executable programs, and other verification tools can complement language models. This focus connects explainable AI with educational STEM tasks, where correctness depends on both the answer and the path used to obtain it.

We introduce **DAPPER**, a task-aware divide-and-plan design for the EXACT 2026 logic and physics tasks. The approach separates semantic formulation, verification preparation, and answer rendering while preserving task-specific representations and verification procedures. Its central principles are that the LLM does not directly determine final correctness, natural language is converted into structured intermediate states, and answer correctness is kept distinct from reasoning artifacts. Our contributions are: (i) a task-aware design for logic and physics educational QA; (ii) bundle-level canonicalization and question-type strategy planning for multi-premise logic; (iii) a physics pipeline that combines plan-before-code reasoning with SI normalization, retrieval, and guarded execution; and (iv) a correctness and transparency contract that makes the verified process available for explanation and inspection.

2 Related Work

LLM reasoning and planning. Prompting-based methods use intermediate language traces to support complex reasoning. Chain-of-thought prompting, for example, asks an LLM to produce intermediate steps before the final answer [3]. Such traces can be useful for presentation, but they do not provide a fixed interface for checking each step and may diverge from the computation that determines the answer. DAPPER treats intermediate states as structured objects and delegates final checking to task-specific verification procedures.

Logic-based reasoning and NL-to-FOL translation. Neuro-symbolic approaches separate language interpretation from formal inference. NL2FOL and LogicLLaMA study the translation of natural-language statements into first-order logic [4,5], while Logic-LM and LINC use LLMs as semantic parsers before invoking symbolic solvers or theorem provers [6,7]. ProofWriter focuses on explicit proofs over natural-language facts and rules [8]. DAPPER uses the same separation for multi-premise educational queries, but must align premises, questions, and answer options within one local representation before checking entailment or refutation.

Execution-grounded and symbolic-code reasoning. Program-aided methods separate language-based reasoning from numerical computation by generating executable programs. PAL translates a problem into a Python program, while Program of Thoughts uses code as an intermediate representation for numerical reasoning [9,10]. SymCode further develops SymPy-based verifiable code generation for mathematical reasoning [15]. Execution makes computation reproducible, but does not by itself specify how physical quantities, units, or formulas were selected. DAPPER extends this setting with quantity normalization, retrieval-guided planning, guarded repair, and recorded symbolic execution results for the physics explanation.

Pedagogical explainability and learner-facing feedback. In educational settings, explainability is also judged by whether the output helps learners and instructors understand a result and act on it. Afzaal et al. study explainable recommendations and feedback for students’ self-regulated learning, emphasizing that explanations should be informative and actionable rather than merely descriptive [16]. DAPPER addresses a related concern for educational problem solving: its explanations are built from premise support, normalized quantities, solution plans, generated code, and execution records, so the reasoning can be inspected by both learners and instructors.

Explainable educational question answering. Educational QA requires answers that are both correct and inspectable. Explanations generated independently of the underlying computation may be fluent but unsupported. DAPPER addresses this limitation through task-specific reasoning artifacts: formal entailment evidence for logic, and normalized quantities, solution plans, code, and execution results for physics.

DAPPER differs from prior work mainly in how it organizes these components. It combines bundle-level canonicalization, question-type strategy planning, plan-before-code reasoning, retrieval, unit normalization, and guarded execution with iterative repair. Although the two branches use different inference mechanisms, both follow the same principle: task-specific verifiers determine the final answer, while intermediate artifacts support its explanation.

Table 1. Comparison of reasoning capabilities emphasized by representative prior methods and DAPPER. A checkmark denotes a core mechanism explicitly used by the method; “—” denotes that the capability is not a stated component of the cited method.

Method	Logic solver	Code execution	Strategy planning	Unit normalization	Repair
Logic-LM [6]	✓	—	—	—	✓
LINC [7]	✓	—	—	—	—
PAL [9]	—	✓	Limited	—	—
PoT [10]	—	✓	Limited	—	—
SymCode [15]	—	✓	✓	Limited	✓
DAPPER (Ours)	✓	✓	✓	✓	✓

3 Our Proposed Method

3.1 DAPPER Problem Formulation

DAPPER treats educational question answering as a sequence of structured transformations rather than a direct mapping from an input problem to an answer. Let x denote an input problem and y the final system output. The reasoning process is modeled as

$$x \rightarrow z_1 \rightarrow z_2 \rightarrow \cdots \rightarrow z_n \rightarrow y, \quad (1)$$

where early stages perform *divide* by converting natural language into structured states, and later stages perform *plan* by preparing a reasoning procedure before inference. Each z_i is produced by one stage and consumed by the next, with its semantics determined by the task. Exposing these states makes it possible to inspect each transformation before the next stage is applied. The resulting reasoning artifacts include formal representations, normalized quantities, reasoning plans, solver outputs, executable programs, and execution traces.

Under this formulation, the logic task contains natural-language premises $P = \{p_1, p_2, \dots, p_m\}$, a query q , and answer options $A = \{a_1, a_2, \dots, a_k\}$. DAPPER predicts an option $\hat{a} \in A$ and a premise-support set E :

$$f_{\text{logic}}(P, q, A) \rightarrow (\hat{a}, E), \quad (2)$$

where E identifies the premises supporting the prediction. Correctness depends on selecting the correct option, while E must be consistent with the formal representation and symbolic inference. The premise set E is the task-level evidence output; formal representations, entailment or refutation paths, and solver traces are intermediate reasoning artifacts rather than primary task outputs.

The physics task maps a natural-language problem s to a numerical or symbolic answer $\hat{y}$ and unit u :

$$f_{\text{physics}}(s) \rightarrow (\hat{y}, u), \quad (3)$$

where correctness requires both a numerically or symbolically correct result within the allowed tolerance and a unit consistent with the target physical quantity. Extracted quantities, normalized units, solution plans, symbolic programs, and execution results form reasoning artifacts without changing this correctness criterion. Unlike the logic task, whose validity is determined by entailment or refutation, the physics task is grounded in physical quantities and executable symbolic or numerical computation.

Thus, DAPPER separates answer correctness from reasoning transparency. Although the two branches address different types of problems, they follow the same underlying principle: rather than allowing the LLM to reason freely and produce answers directly, the LLM is only responsible for constructing a solution plan or generating an intermediate representation, while the final answer must be validated by a task-specific verifiable reasoning mechanism.

3.2 DAPPER Architecture Overview

Figure 1 illustrates the overall architecture of DAPPER, comprising three main stages: problem analysis (Divide), solution planning (Plan), and verification/execution (Verify/Execute). First, the system standardizes the symbols in the problem statement and classifies the problem as either Type 1 or Type 2 to route it to the corresponding processing pipeline. For Type 1, the system analyzes the problem’s premises, identifies the question type, and selects an appropriate verification method before passing logical propositions to the Z3 solver. For Type 2, the system extracts quantities, units, and relevant information, then retrieves the necessary physics knowledge from the knowledge base to formulate a solution plan. In the final stage, each branch employs a specific verification mechanism to ensure the correctness of the result. Specifically, Type 1 uses Z3 to verify logical propositions, while Type 2 utilizes symbolic execution to execute and verify the calculation process. Once the answer is confirmed, the system generates an explanation based on intermediate results and information recorded during the reasoning process before returning the final output.

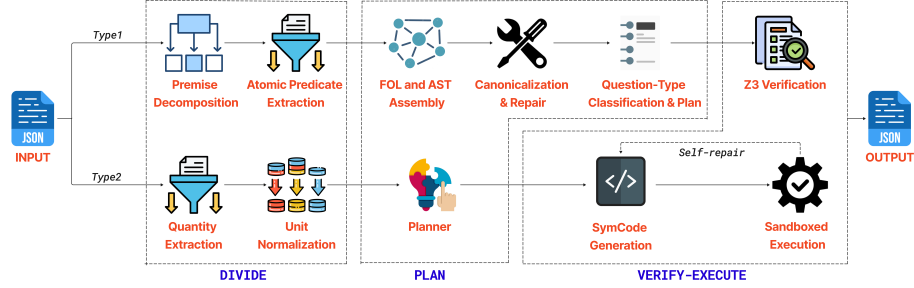


Fig. 1. DAPPER architecture for task-specific logic and physics reasoning.

3.3 Type 1: Logic Reasoning Pipeline

Type 1 uses a solver-aided workflow that converts a logic question into claims that can be checked by the Z3 SMT solver [13]. Let $P = \{p_i\}_{i=1}^m$ denote the premises, q the question, and $A = \{a_j\}_{j=1}^k$ the answer choices. DAPPER maps the premises to a joint formula Φ_P and uses q together with each candidate answer to construct a target formula ϕ_j . A candidate is entailed when $\Phi_P \models \phi_j$ and refuted when $\Phi_P \models \neg\phi_j$. The language model therefore interprets the wording and proposes the formal structure, whereas the solver determines which formal claim follows from the premises.

The conversion begins by decomposing each premise into typed semantic parts. For example, “every student who passes logic is eligible” is represented using a variable x , a restrictor “student”, a condition “passes logic”, and a conclusion

“eligible”. The representation also records facts, numeric or temporal constraints, and modality when they occur. Lightweight checks handle common framing errors, such as treating a numeric fact as a rule or an equivalence as a one-way implication. Statements such as “not necessarily” are retained as witnesses rather than forced into ordinary classical FOL.

The extracted phrases are then mapped to atomic predicates and assembled into a logical abstract syntax tree (AST). In the running example, “student” and “passes logic” become unary predicates over x , while “eligible” forms the consequent. Repeated phrases reuse the same predicate name across premises. The AST represents negation explicitly and stores statements such as “score is at least 60” as comparison nodes. DAPPER renders the FOL string from this tree and checks its structure before passing it to Z3, so the solver expression remains tied to the parsed representation rather than being generated as unconstrained text.

Because Z3 treats different predicate names as unrelated symbols, DAPPER performs bundle-level canonicalization before verification. Linguistic variants such as “eligible” and “can enroll” are aligned when they refer to the same property, and the same treatment is applied to entity names and aliases. The resulting bundle gives the premises and candidate claims a shared vocabulary, which is required for the subsequent entailment checks.

The final preparation step is strategy planning. DAPPER classifies the question structure and selects the corresponding answer procedure: direct entailment checks each option independently; scenario-based reasoning adds local scenario conditions as temporary premises; and evidence-oriented questions, including fewest-premises and premise-tracking questions, search for the premise subset or proof path associated with a candidate. For yes/no questions, both the query and its negation are checked to distinguish entailment, refutation, and uncertainty. Z3 returns the status for each tested claim, and the system selects the answer from these statuses. At the task-output level, Type 1 returns the selected option $\hat{a}$ and the premise-support set E ; the formal premises, AST, candidate formulas, solver statuses, and proof paths are retained as reasoning artifacts used to generate the explanation.

3.4 Type 2: Physics Reasoning Pipeline

Physics question answering requires both selecting an appropriate physical model and carrying out its calculation reliably. A language model may identify the relevant theory but still fail because of arithmetic mistakes, unit mismatches, or invalid algebraic roots. DAPPER therefore uses the LLM to construct the solution procedure and delegates the calculation to symbolic execution. As shown in Figure 1, the Type 2 pipeline consists of information extraction, unit normalization, formula retrieval, solution planning, code generation, and sandboxed execution.

The process begins with an LLM parsing the problem into a structured JSON state. This state records the given quantities and their original units, the target quantity, and relevant spatial or temporal constraints, such as direction or

geometric relations. The remaining stages operate on this representation rather than repeatedly interpreting the original statement. This separation reduces the burden on the model and keeps problem understanding distinct from formula selection and calculation.

The extracted quantities are converted to SI units and assigned standardized ASCII labels. Metric prefixes, Unicode symbols, and equivalent notations such as m/s are resolved before code generation. The normalized state is then used to retrieve relevant “solver cards” from a physics database following retrieval-augmented generation principles [11]. Each card contains formulas, physical constants, SymPy templates, sign conventions, and common unit-handling pitfalls. Although the retriever supports dense representations such as E5 embeddings [12], the submitted configuration uses lexical retrieval. The retrieved cards provide the physical context for the solution plan and help identify relevant equations and edge cases.

The plan and normalized quantities are passed to the SymCode generation stage. The generated Python program uses SymPy [14] to formulate the equations, solve them symbolically, discard physically invalid solutions such as negative time, and compute the target value. It returns a numerical result together with its unit. The program is then executed in a sandbox that blocks unsafe operations, including file writing and network access, and enforces a four-second timeout. If execution fails, the traceback and the original context are returned to the model for guarded code repair, and each repair iteration is logged.

After successful execution, the system generates a natural-language explanation from the problem statement, normalized quantities, retrieved solver cards, solution plan, generated code, and execution logs. The explanation is thus linked to the artifacts and computation that produced the answer rather than to an independent prose-only reasoning trace.

The experiments in Section 4 evaluate the complete submitted pipeline on the official benchmark.

4 Experiments and Official Results

4.1 Implementation Details

We use Qwen3-8B [2] for all language-model stages in both branches. The model runs with 8-bit quantization. It handles semantic parsing, strategy or solution planning, code generation, and explanation, while the final logic checks are performed by Z3 [13] and the physics calculations by a sandboxed SymPy interpreter [14]. Thus, the submitted system does not use the language model itself to perform the final logical decision or numerical calculation.

4.2 Official Leaderboard Results

In the final public evaluation round, the submitted system processed all 50 queries and placed second on the EXACT 2026 leaderboard. Type 2 received

25/25 points: all physics answers met the numerical and unit requirements after sandboxed execution and self-repair. Type 1 received 23.91/25 points, corresponding to 95.64% for answer selection and premise-support identification. In the separate private human evaluation, the explanations received a score of 3.2/4 from the organizers and participating professors. These results contributed to the team being selected among the top teams in EXACT 2026.

Table 2. Top-10 teams in the official public EXACT 2026 Round 2 evaluation.

Rank	Team	Raw points	Type 1	Type 2	Time
1	BKcodex1	52.95	23.87	25.00	4.07
2	lqd77bd (Ours)	52.89	23.91	25.00	3.98
3	lal	52.79	23.95	25.00	3.84
4	Laplaces-red-devils	52.64	24.32	24.00	4.32
5	The Scourge	52.42	24.47	24.00	3.95
6	CoTu	52.20	23.10	25.00	4.10
7	hi_fine	50.95	24.21	23.00	3.74
8	Solo	50.84	22.91	24.00	3.93
9	A2PK	50.59	21.85	25.00	3.74
10	AltF4	50.31	23.29	23.00	4.03

4.3 Pipeline-Level Ablation

We conduct a pipeline-level ablation using the 1,500-instance physics test dataset provided by the EXACT 2026 organizers. All configurations are evaluated on the same test data using the same Qwen3-8B backbone [2] and evaluation procedure. We compare direct LLM reasoning, the SymCode baseline [15], and the complete DAPPER pipeline.

Table 3. Pipeline-level ablation on the organizer-provided physics test dataset.

Method	Accuracy (%)
Direct LLM	42.56
SymCode	70.09
Full DAPPER	83.15

As shown in Table 3, Direct LLM reasoning achieves an accuracy of 42.56%. SymCode improves accuracy to 70.09%, an absolute gain of 27.53 percentage points. This improvement demonstrates the benefit of generating and executing symbolic programs instead of relying on the LLM to perform numerical calculations directly.

Full DAPPER further improves accuracy to 83.15%, outperforming SymCode by 13.06 percentage points. This result shows that incorporating structured information extraction, solver-card retrieval, and explicit solution planning further improves physics reasoning performance beyond SymCode alone.

5 Conclusion and Limitations

5.1 Limitations and Future Work

Despite its strong performance, DAPPER’s evaluation is primarily focused on the EXACT 2026 benchmark, and broader validation on diverse public datasets is needed to assess the generalizability of the divide-and-plan approach. The physics retriever also operated in lexical-only mode for this submission, leaving its hybrid dense-lexical design partly untested. Finally, although answer accuracy was high, the human explainability score of 3.2/4 indicates that the generated explanations, while grounded in verified computation, still leave room for improvement in clarity.

5.2 Concluding Remarks

We presented DAPPER, a task-aware divide-and-plan design for explainable educational question answering, separating semantic formulation, verification preparation, and answer rendering while grounding final answers in task-specific formal inference or executable computation. DAPPER combines Z3 checking for logic with safety-constrained SymPy execution for physics. This produces inspectable reasoning artifacts alongside validated responses. In the official Round 2 evaluation, the system ranked second overall and achieved a perfect score on the Type 2 physics track.

These results indicate that a compact model such as Qwen3-8B can produce reliable and inspectable outputs when paired with explicit, verifiable reasoning stages. This combination may support educational QA deployments under the resource constraints of real classrooms and other low-resource settings without sacrificing explainability.

The divide-and-plan approach was evaluated on two specific tasks, but its underlying principle of splitting reasoning into checkable stages is not limited to these settings. Applying the approach to other explainable AI tasks remains an avenue for future work.

References

1. Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al.: A Survey of Large Language Models. arXiv preprint arXiv:2303.18223 (2023)
2. Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al.: Qwen3 Technical Report. arXiv preprint arXiv:2505.09388 (2025)

3. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q.V., Zhou, D.: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837 (2022)
4. Lalwani, A., Chopra, L., Hahn, C., Trippel, C., Jin, Z., Sachan, M.: NL2FOL: Translating Natural Language to First-Order Logic for Logical Fallacy Detection. *arXiv preprint arXiv:2405.02318* (2024)
5. Yang, Y., Xiong, S., Payani, A., Shareghi, E., Fekri, F. Harnessing the Power of Large Language Models for Natural Language to First-Order Logic Translation. *Proceedings of ACL 2024*, pp. 6942–6959 (2024)
6. Pan, L., Albalak, A., Wang, X., Wang, W.Y.: Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824 (2023)
7. Olausson, T.X., Gu, A., Lipkin, B., Zhang, C.E., Solar-Lezama, A., Tenenbaum, J.B., Levy, R.: LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. *arXiv preprint arXiv:2310.15164* (2023)
8. Tafjord, O., Dalvi, B., Clark, P.: ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634 (2021)
9. Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., Neubig, G.: PAL: Program-Aided Language Models. In: *Proceedings of the 40th International Conference on Machine Learning*, PMLR, vol. 202, pp. 10764–10799 (2023)
10. Chen, W., Ma, X., Wang, X., Cohen, W.W.: Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023)
11. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020)
12. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., Wei, F.: Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533* (2022)
13. de Moura, L., Bjørner, N.: Z3: An Efficient SMT Solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) *TACAS 2008*. LNCS, vol. 4963, pp. 337–340. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24
14. Meurer, A., Smith, C.P., Paprocki, M., Čertík, O., Kirpichev, S.B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J.K., Singh, S., et al.: SymPy: Symbolic Computing in Python. *PeerJ Computer Science* **3**, e103 (2017). <https://doi.org/10.7717/peerj-cs.103>
15. Bagheri Nezhad, S., Li, Y., Agrawal, A.: SymCode: A Neurosymbolic Approach to Mathematical Reasoning via Verifiable Code Generation. In: *Findings of the Association for Computational Linguistics: EACL 2026*, pp. 1489–1503. Association for Computational Linguistics (2026). <https://doi.org/10.18653/v1/2026.findings-eacl.76>
16. Afzaal, M., Zia, A., Nouri, J., et al.: Informative Feedback and Explainable AI-Based Recommendations to Support Students’ Self-regulation. *Technology, Knowledge and Learning* **29**, 331–354 (2024). <https://doi.org/10.1007/s10758-023-09650-0>